Mini-Project: 18

Bachelor of Computer Application
Semester – IV

Sub: Full Stack Development

Topic: SMS API

By
Name: Prem Panchal
Reg no.: 2411021240011

Faculty Name: VEERA RAGHAV K

Faculty Signature: _____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106

March 2026

Mini Project - 18: SMS API

Introduction

This project demonstrates how to build a basic REST API using Node.js, Express.js, and MongoDB to manage student data. The API allows users to perform CRUD operations such as creating, retrieving, updating, and deleting student records.

MongoDB is used as the database to store student information, while Express.js handles API routes and server functionality. Postman is used to test API endpoints.

Objectives

The main objectives of this project are:

- To understand backend development using Node.js and Express
- To connect a MongoDB database with Node.js
- To perform CRUD operations using REST API
- To test API endpoints using Postman

Tools & Technology used

- **Express.js :-** For web framework
- **MongoDB :-** For database
- **Postman :-** For API Testing
- **VS Code :-** Code Editor
- **MongoDB Compass :-** For Database GUI

Project Setup (Commands Used)

Commands executed in CMD :-

mkdir sms-api : makes a folder named sms-api
cd sms-api : fetches the sms-api folder
npm init -y :

Create files:

server.js
Student.js

MongoDB Database Setup

- **Default Connection URL :** mongodb://localhost:27017/studentdb

Database Creation Steps:

1) Install MongoDB
2) Start MongoDB
3) Connect MongoDB Compass
4) Run the node.js server
5) Insert student data using Postman

Project Structure

→ **sms-api:**

- Student.js
- Server.js

Student.js Code

```
const mongoose=require("mongoose");
const StudentSchema=new mongoose.Schema({
   name:String,
   age:Number,
   course:String});
module.exports=mongoose.model("Student",StudentSchema);
```

Server.js Code

```
const mongoose=require("mongoose");
const express=require("express");
const Student=require("./Student");
const app=express();
app.use(express.json());

// MONGO DB CONNECTION
mongoose.connect("mongodb://localhost:27017/studentdb")
.then(()=>console.log("MongoDB Connected"))
.catch(err=>console.log(err));

//
app.post("/students", async (req, res) => {
   try {
      const students = await Student.insertMany(req.body);
      res.send(students);
   } catch (err) {
```

```
      res.send(err);
   }
});

//get student
app.get("/students",async(req,res)=>{
   const students=await Student.find();
   res.send(students);
});

// Update students(PUT)
app.put("/students/:id",async(req,res)=>{
   const student = await Student.findByIdAndUpdate(
      req.params.id,
      req.body,
      {new:true}
   );
   res.send(student);
});

// Delete
app.delete("/students/:id",async(req,res)=>{
   const student=await Student.findByIdAndDelete(req.params.id);
   res.send(student)
});
```

```
// cresting local host server
app.listen(3000,()=>{
   console.log("http://localhost:3000/")

});
```

Running the code in the terminal:

Starting the server: node server.js

If Successful : MongoDB Connected will be displayed

API Testing using POSTMAN :

1) **POST Request:**
   URL:- http://localhost:3000/students

   Body (JSON):

```
[
{ "name": "Mahir", "age": 20, "course": "BCA" },
{ "name": "Prem", "age": 19, "course": "BCA" },
{ "name": "Piyush", "age": 22, "course": "BCA" },
{ "name": "Riti", "age": 19, "course": "BCA" },
{ "name": "Tejas", "age": 19, "course": "BCA" },
{ "name": "Dakshraj", "age": 19, "course": "BCA" },
]
```

   Inserts the data into the MongoDB

2) **GET Request:**
   URL:- http://localhost:3000/students
   The retrieves all student records

3) **PUT Request:**
   URL :- http://localhost:3000/students/:id
   The updates the record

4) **DELETE Request:**
   URL :- http://localhost:3000/students/69b24169d2b14dd0f74eba3e
   This deletes a student record

Browser Output

When the GET API is opened in the browser:

http://localhost:3000/students

The browser displays student records in JSON format.

Example:


```
[
 {
  "_id": "65d3a1c5c12c",
  "name": “Tilak",
  "age": 19,
  "course": "CA"
 }
]
```


/students API Response

After inserting all 10 students, the GET request returns:


```
[
 {
  "_id": "1",
  "name": "Mahir",
  "age": 20,
  "course": "BCA"
```

},
{
"_id": "2",
"name": "Prem",
"age": 19,
"course": "BCA"
}
]

The response contains all student records stored in the MongoDB database.

MongoDB Compass Database View

Steps to view data:

1. Open MongoDB Compass
2. Connect using:

mongodb://localhost:27017

3. Open database:

studentdb

4. Open collection:

students

Results / Output

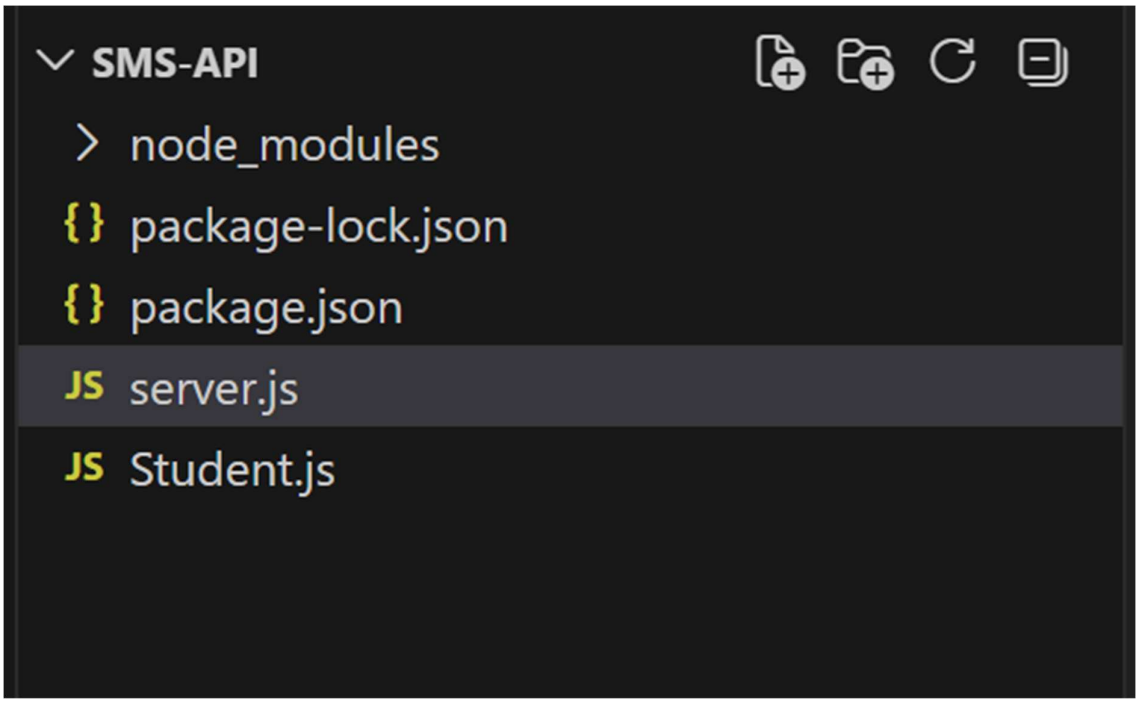
- The Node.js server successfully connected to MongoDB.
- Student data was inserted using POST API.
- Data was retrieved using GET API.
- Student data could be updated using PUT API.
- Student records could be deleted using DELETE API.
- All data was stored and visible in MongoDB Compass.

Conclusion

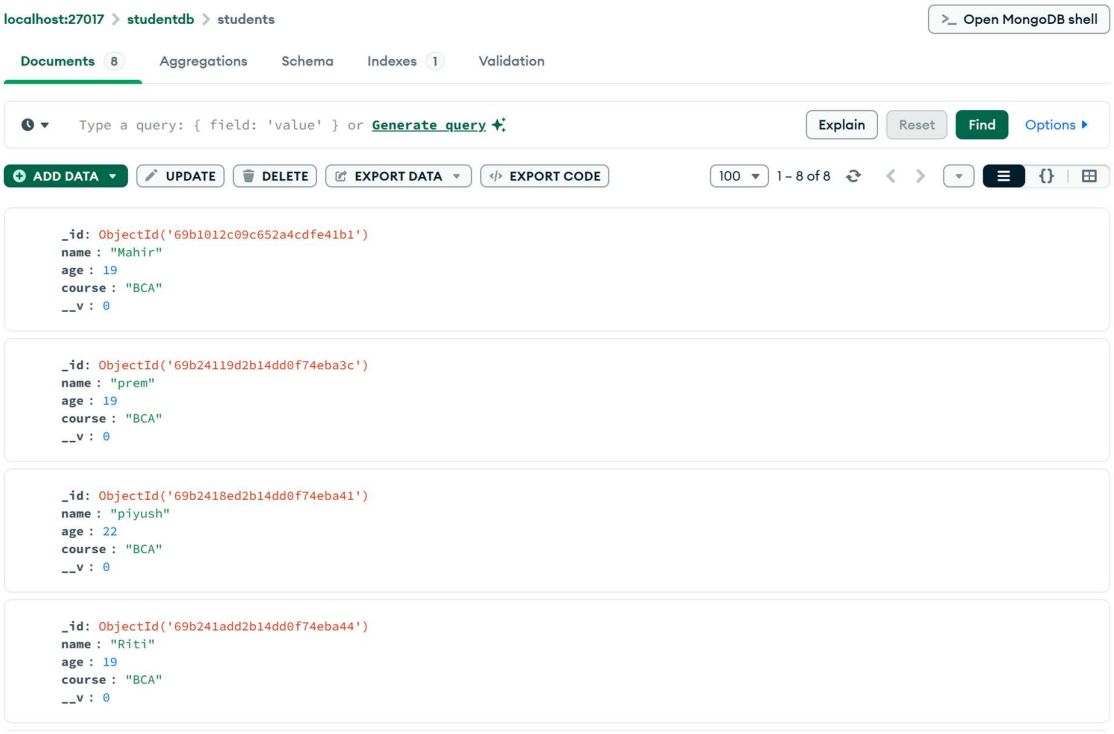
This project successfully demonstrates how to create a REST API using Node.js, Express, and MongoDB. It shows how CRUD operations can be implemented and tested using Postman.

The project helps in understanding backend development concepts such as API creation, database integration, and server management.

VS Code File Structure



MongoDB Compass Database View



Browser GET View